What is an array?

An array is an ordered collection of values stored under one name. Each value is called an element, and its position is identified by an index.

A small C++ example

```
int scores[5] = {88, 94, 76, 100, 91};
scores[0] is 88; scores[4] is 91. Indexing starts at 0.
```

Why are arrays needed?

- **Keep related data together:**
  One collection is easier to manage than many separate variables.
- **Use loops:**
  A loop can process every element for totals, searches, or reports.
- **Access by position:**
  An index gives direct access to an element when its position is known.

A couple of things to remember

Indexes start at 0, so an array of 5 elements has positions 0 through 4. Using a position outside that range is an error to avoid.

A built-in array or C++ std::array has a fixed size. Use std::vector when the number of items needs to grow or shrink while the program runs.

Arrays in everyday software

Arrays are useful whenever software needs an ordered group of similar values. Examples include:

Weather apps

Keep a day’s hourly temperatures together to calculate a daily high or average.

Music players

Store an ordered playlist so tracks can be shown and played one after another.

Games

Keep player scores together to sort or display a leaderboard.

Photo software

Represent image pixels as rows and columns of color values.

Shop inventory

Keep product records together to search products and total their stock value.

Project: Array Inventory Tracker

The companion C++ console app models a small shop. It stores up to 20 products in std::array<Product, 20>. Each array element is a Product record with a name, quantity, and unit price.

```
1. Add: put a new product into the next unused array position.
2. List: loop through stored products and display their details.
3. Find or update: compare product names while looping over the array.
4. Total: multiply each quantity by its price and add the results.
```

A count records how many of the 20 positions are in use. Before adding, the app checks that there is room; its loops only visit used positions. Real shops need flexible storage and reliable saving, so production software commonly uses resizable collections and databases instead.